53. Capstone: Harden and Operate a Linux Web Server

53. Capstone: Harden and Operate a Linux Web Server

Phase: Phase 12 — Capstones
Estimated time: 3–5 days
Slug: `53-capstone-server`

Learning objectives

- Provision a Linux VM
- Serve a simple site behind a reverse proxy
- Add firewall, SSH hardening, systemd unit
- Document runbook + restore notes

Prerequisites

Complete prior numbered lessons. Prefer a real Linux VM/container for Linux-specific topics. Type every command yourself — muscle memory beats skimming.

Why this matters

This capstone ties the course together. You will build a small server the way a careful operator would: it comes up after a reboot, serves traffic, accepts administration only from you, exposes the minimum surface, and has a runbook so someone else could take over. Passing means you can show evidence for each claim, not just say “it works”.

The exposed surface is your first proof. A web server with SSH needs exactly three TCP ports reachable from outside: 22 for SSH, 80 for HTTP (usually only to redirect), and 443 for HTTPS. Anything more must be justified in writing.

Core concepts

Layer	Deliverable	Evidence
Access	key-only SSH, no root login	`sshd -T`, failed password test
Network	default-deny firewall: 22, 80, 443	`nft list ruleset`, external scan
Service	systemd unit for the app, hardened	`systemd-analyze security`
Edge	nginx or Caddy with TLS	`curl -vk`, cert dates
Data	backup and restore drill	restore diff output
Observability	logs and basic metrics	`journalctl`, node exporter
Operations	runbook, reboot test	notes, uptime after reboot

```
internet
   |  22 / 80 / 443 only
[ firewall: policy drop ]
   |
[ nginx :443 TLS ] --proxy--> [ app :8080 on 127.0.0.1, User=app ]
   |                                |
/var/log/nginx              journald, /var/lib/app (RW), backups
admin: ssh key -> sudo -> systemctl, journalctl, runbook.md
```

Worked examples

Example 1: baseline and plan.

```
hostnamectl
ip -br a
sudo ss -tlnp
sudo useradd -m -s /bin/bash deploy
```

Record every listening port before you change anything.

Example 2: SSH hardening, with a second session open.

```
sudo tee /etc/ssh/sshd_config.d/10-hardening.conf <<'EOF'
PasswordAuthentication no
PermitRootLogin no
EOF
sudo sshd -t && sudo systemctl reload ssh
sudo sshd -T | grep -E 'passwordauthentication|permitrootlogin'
```

Example 3: firewall allowing only the required set.

```
sudo ufw default deny incoming
sudo ufw allow 22/tcp
sudo ufw allow 80/tcp
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status numbered
```

Example 4: bind the app to loopback and let the proxy front it.

```
sudo ss -tlnp | grep 8080     # expect 127.0.0.1:8080, not 0.0.0.0
curl -sI http://127.0.0.1:8080/ | head -1
curl -skI https://localhost/ | head -1
```

Example 5: backup and restore evidence.

```
sudo tar -C /var/lib -czf /tmp/app-backup.tgz app
mkdir -p /tmp/restore && tar -C /tmp/restore -xzf /tmp/app-backup.tgz
diff -r /var/lib/app /tmp/restore/app && echo RESTORE_OK
```

Mini-lab

1. Provision a fresh VM and snapshot it.
2. Write the baseline: listening ports, users, services.

3. Install nginx, deploy a static page or proxy a small app on loopback.
4. Add a TLS certificate (self-signed is fine) and test with `curl -vk`.
5. Harden SSH, then the firewall; verify with an external scan or `nc`.
6. Write the systemd unit with `User=`, `NoNewPrivileges=true`, and `ProtectSystem=strict`.
7. Run the backup and restore drill; save the output.
8. Reboot. Verify everything returns without manual steps.
9. Write `runbook.md`: start, stop, logs, restore, rollback.

Decision guide / failure modes

Question	Answer
Which ports remain open?	22, 80, 443; justify any extra
App listens where?	127.0.0.1 behind the proxy
Password SSH?	off; keys only
Proof of restore?	recorded diff or checksum result
Survives reboot?	unit enabled, firewall persisted

Failure modes: ClickOps with no notes; ports open to the world because the app bound to 0.0.0.0; firewall locking you out; certificates that expire silently; no reboot test; and a runbook written from memory instead of from the commands you actually ran. Reproducibility is the rubric.

Common mistakes

- ClickOps without notes
- Ports 0.0.0.0 wide open
- No reboot test

Exercises

Work in `exercises/53-capstone-server/`. Read that folder's `README.md` fully before coding.

```
cd exercises/53-capstone-server
chmod +x test.sh exercise.sh
./test.sh
```

What you will build

Open `exercise.sh` and implement `checklist_min_ports`. It prints the comma-separated list of TCP port numbers, in ascending order and without spaces, that a hardened web server with remote administration should expose to the internet: one for SSH, one for plain HTTP, and one for HTTPS. One line, nothing else. After it passes, build the capstone and produce an evidence folder with the listening ports, firewall status, `sshd -T` excerpt, certificate dates, restore diff, and post-reboot checks. Any port beyond these three needs a written reason.

Study loop

- **A (Drill):** make `./test.sh` pass
- **B (Apply):** run 2–3 lab commands from this lesson on a real Linux host; paste notable output into your notes
- **C (Challenge):** rewrite the exercise from memory the next day

Do not open `solutions/53-capstone-server/` until you have an honest failing attempt.

Self-check

Try to answer without scrolling up:

1. What ports are open and why?
2. How prove restore?
3. What happens after reboot?

Further reading

- https://ubuntu.com/server/docs